

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Taller de Diseño Digital (CE3201)

Laboratorio 2 - Informe

Lógica Combinacional y Aritmética

Segundo Semestre

Profesor:

Luis Alonso Barboza Artavia

Estudiantes:

Mauro Andrés Navarro Obando | 2018170753

Yonathan Monge Sanabria | 2019308543

Grupo 01

Fecha de entrega: miércoles 10 de setiembre

Año 2025

Abstract

Este informe presenta el diseño e implementación de una Unidad Aritmético-Lógica (ALU) de 4 bits utilizando lógica combinacional y circuitos aritméticos como parte del Laboratorio de Diseño Digital. El trabajo aborda la síntesis de operaciones aritméticas fundamentales como suma, resta y multiplicación, destacando el uso de sumadores completos, sumadores en cascada (ripple-carry adders) y operaciones de desplazamiento a la izquierda (shift-left). Asimismo, se discute la importancia de la unidad de control, los tiempos de propagación y de contaminación, así como la integración de flags de estado (Zero, Carry, Negative y Overflow) para mejorar la funcionalidad de la ALU. El diseño propuesto fue implementado y simulado en una plataforma FPGA para validar el comportamiento correcto de los circuitos. Los resultados demuestran que la ALU sintetizada ejecuta correctamente las operaciones planteadas, confirmando la viabilidad del enfoque combinacional en el diseño aritmético digital.

Keywords

Unidad Aritmético-Lógica (ALU), sumador en cascada, sumador completo, resta, multiplicación, FPGA, lógica combinacional, *flags* de estado, diseño digital.

Introducción

¿Dónde está realmente el corazón de una computadora? Las computadoras actuales alcanzan un nivel tan grande de complejidad que puede ser complicado definir exactamente cuál es su corazón, pero después de hacer un análisis de una arquitectura y comprender que al final una computadora basa todo su funcionamiento e cálculos concretos y

exactos sobre un sistema numérico binario, es más fácil dilucidar que el corazón de la computadora es el CPU y dentro de este es la Unidad Aritmético-Lógica la que se encarga de procesar todos los datos que fluyen por un computador y se encarga de que el software pueda correr y procesarse correctamente y que se proporcione una experiencia constate a los usuarios.

La ALU en sí es como ver una calculadora optimizada al máximo que trabaja en base binaria. Las operaciones que realiza una ALU están minuciosamente estructuradas mediante un diseño que ha pasado por un proceso de **síntesis lógica**; de modo que la ALU ejecuta todas las operaciones mediante una serie de circuitos combinacionales complejos que se conocen como **circuitos aritméticos**. Asimismo, al trabajar sobre lógica combinacional entendemos que las salidas solo dependen de las entradas de la ALU y no de estados previos; lo cual tiene sentido ya que la ALU genera los cálculos “sobre la marcha” o en tiempo de ejecución.

La ALU por sí sola no podría funcionar adecuadamente y sería muy ineficiente, es por esto que la ALU trabaja en conjunto con otras estructuras tales como la **unidad de control**, los **registros** y la **capa de caché**. No todas estas estructuras secundarias son necesarias para una ALU básica, pero es la manera en que

se estructuran muchos sistemas actuales.

La ALU como tal consiste en compuertas lógicas las cuales están agrupadas de una manera tal que ciertos grupos de compuertas pueden ejecutar operaciones matemáticas sobre una de bits determinada y pasar la salida a otro conjunto de compuertas que **representa otra operación matemática.**

Sin embargo, el objetivo de realizar todas estas operaciones generalmente es controlado por una unidad superior que es la que lleva el orden en la que estas operaciones y sus salidas deben ser ejecutadas para que el proceso cobre sentido lógico. Esta sección que controla el orden de las operaciones de la ALU es la **unidad de control.**

“La unidad de control (UC) es como el administrador de la CPU: no realiza el procesamiento real, pero se asegura de que todo funcione en orden.” [1]. Algunas de los controles que realiza la UC son: enviar señales específicas para escribir o leer información de los registros, indica la operación actual a ejecutar, habilita la lectura y escritura a memoria, entre otras acciones.

Hay un concepto que es de suma importancia en el desarrollo de una ALU eficiente, que es la sincronidad ya que, como ya se dijo, todas las operaciones deben de llevar un orden específico para que no choquen las operaciones, o no se realice un cálculo sin que los circuitos aritméticos

previos hayan brindado su respectiva salida. Para esto al momento de diseñar hay que considerar los conceptos de **tiempo de propagación** y **tiempo de contaminación.**

Los tiempos de contaminación y propagación son dos conceptos importantes en tomar en cuenta. El tiempo de contaminación es el mínimo tiempo que puede tardar la salida en empezar a cambiar después de un cambio en la entrada. Y el tiempo de propagación es tiempo que tarda la salida en reflejar un cambio de la entrada. Se mide desde que la señal de entrada cruza el 50% de su valor hasta que la salida cruza también el 50%. Estos dos conceptos son importantes debido a que la electrónica real, no la ideal, no tienen tiempos de reacción “perfectos”, ósea que de 0 a 1 no es una transición instantánea y se debe pasar de 0 a 1 en una cantidad de tiempo tangible.

Dentro del presente documento se explicará la manera de diseñar algunas operaciones para una ALU y como implementar todos estos circuitos aritméticos para que en conjunto puedan realizar operaciones más complejas. Además, se explora la implementación de estas operaciones en la FPGA para poder modelar y experimentar mejor los comportamientos reales de estos circuitos.

Desarrollo

Ejercicio 1

Este primer ejercicio nos indica dos puntos clave, diseñar una ALU que contenga **flags**, y un total de **10 operaciones aritméticas**. Además, se pide **implementar una calculadora** que maneje cantidades de 4 bits, es decir llegaremos hasta el 1111 o el 15 en decimal.

→ Diseño

Circuitos básicos para operaciones elegidas

Suma:

Para lograr hacer una suma completa de N bits, se propone el uso de un Full-Adder, su funcionamiento se basa en la siguiente tabla de verdad:

A	B	C_{in}	C_{out}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Dónde A, B son los bits de entrada que se suman, C_{in} es el bit del posible carry de la suma anterior, por otra parte, S representa la salida resultante de la suma entre A y B, C_{out} es el bit carry resultante de la suma entre A y B.

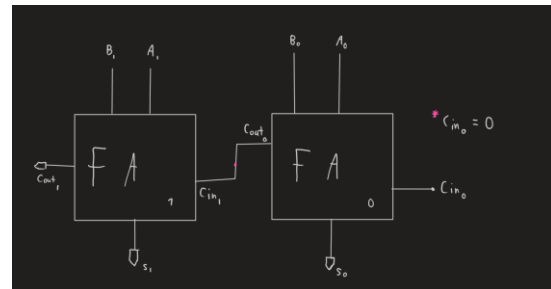
La ecuación booleana del comportamiento para la salida del bit S es el siguiente

$$S = (A \oplus B) \oplus C_{in}$$

La ecuación booleana del comportamiento para la salida del bit C_{out} es el siguiente:

$$C_{out} = (A \text{ or } B) \text{ or } [(A \oplus B) \text{ and } C_{in}]$$

El siguiente diagrama ejemplifica la conexión de los Full-Adder:

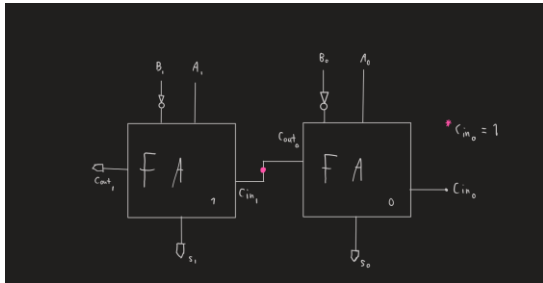


Resta:

En el caso de la resta se realiza un ajuste al sumador completo anterior (Full-Adder), dicho ajuste se trata de negar todos los bits de entrada del número B e inicializar el bit menos significativo del carry con un valor de 1 ($C_{in} = 1$), se debe aclarar que el B representa el número que sustrae (sustraendo), esto permite sumarle un bit al complemento del número B, la tabla de verdad del funcionamiento del restador completo es de la siguiente forma:

A	B	C_{in}	C_{out}
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Diagrama del restador completo:



Multiplicación:

Para la multiplicación se ha tomado la funcionalidad de la operación (shift-left) que funciona para multiplicar un número por alguna potencia de 2. La cantidad de movimientos a la izquierda (shift-left) que se apliquen a un número binario es la equivalente a: el número(2^n movimientos)

$$\begin{array}{l}
 3 \\
 0011 \ll 2 = 3 \cdot 2^2 = 12 \\
 1100 = 12 \equiv V_0
 \end{array}$$

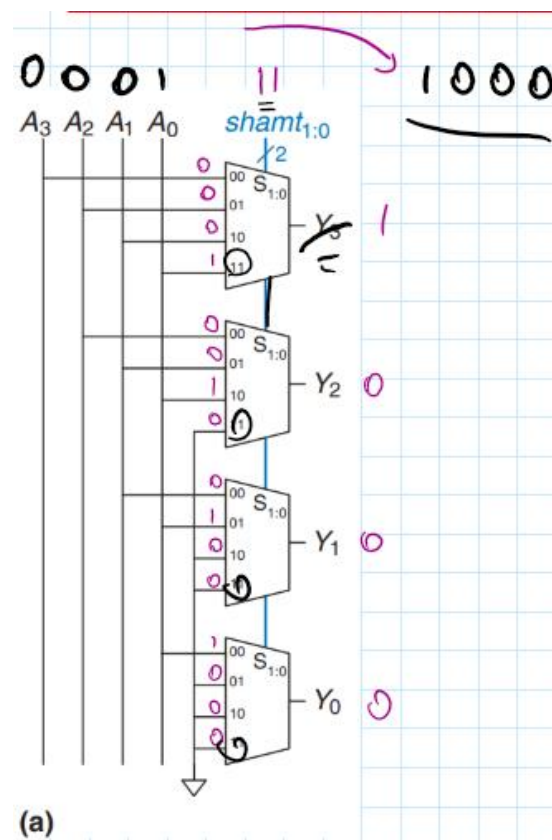
Ilustración 1

En la *ilustración 1* podemos ver como al 3 se le está aplicando un doble left-shift; por tanto, al aplicar un left-shift nos queda el número: 0110 y al aplicar un segundo shift nos queda el número:

1100. Según la teoría un doble left-shift equivalente a multiplicar el número original por 2^n y podemos apreciar que efectivamente:

$$\begin{array}{l}
 3 \cdot 2^2 = 12 \\
 1100 = 12_{10}
 \end{array}$$

Un left-shifter se puede aplicar por medio de multiplexores, de modo que un left-shifter que maneje números de 4-bits se verá de la siguiente manera:



Aplicando shifters y sumas, podemos multiplicar 2 cantidades como se muestra en el siguiente ejemplo:

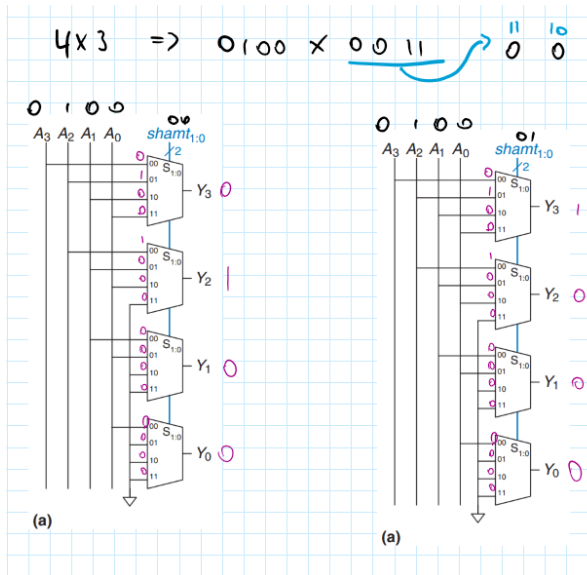


Ilustración 2

En la ilustración 2 se retoma el ejemplo de realizar la operación 4 x 3 la cual es igual a 12 y en binario la operación se ve como:

$$0100 \times 0011 = 1100$$

El 3 se descompone en potencias de 2 como $2^1 + 2^0$ lo que es posteriormente representado al convertir el 3 a binario en el propio número 0011. Se le da un ID a cada bit del número 0011 y los que quedan encendidos en 1 son las entradas de los multiplexores usan para determinar cuántos shift-left realizar al número original. Finalmente se suman los resultados y se verifica que la suma es correcta:

$$\begin{array}{r} 0100 \quad (4 \text{ en decimal}) \\ + 1000 \quad (8 \text{ en decimal}) \\ \hline 1100 \quad (12 \text{ en decimal}) \end{array}$$

Con esto vemos la manera en la que se pueden implementar 3 de las operaciones básicas de una ALU.

Ahora una ALU consta de otras operaciones como AND, OR, división, módulo, etc. Además, las ALU tienen otras maneras de comunicar información al usuario, esto se hace por medio de **flags**. Los flags con otras salidas que en realidad son bits, los cuales expresan datos extra sobre las operaciones generadas, como lo puede ser la flag de Zero, Carry, Negative y Overflow. Estas flags permiten a la Unidad de Control tomar ciertas consideraciones y decisiones dependiendo del resultado de las flags.

Con todo esto, podemos considerar como base el diseño simple pero conciso presentado en Harris & Harris, como se muestra en la Fig. 2 [2].

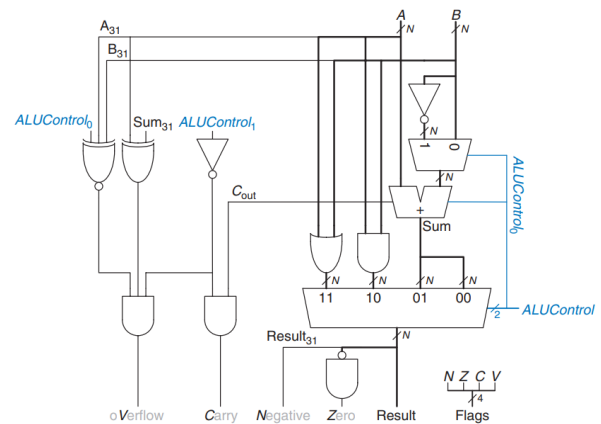


Fig. 2. Diseño de una ALU básica tomado de Harris & Harris [2].

- Solución Practica:

Luego para la elaboración de la calculadora, se realiza la

implementación de las 10 operaciones indicadas.

Vamos a iniciar con la suma. La suma la vamos a generar por medio de un ripple-carry adder para 4 bits, vamos a utilizar las ecuaciones propuestas para un full adder y luego concatenamos 4 full adders:

```
module full_adder(
    input logic A,
    input logic B,
    input logic Cin,
    output logic S,
    output logic Cout
);

    logic axb; // A XOR B

    // XOR intermedio
    assign axb = A ^ B;
    assign S = axb ^ Cin;

    // Carry out
    assign Cout = (A & B) | (axb & Cin);

endmodule

module aluDesign(
    input logic [3:0] A,
    input logic [3:0] B,
    input logic Cin,
    output logic [3:0] S,
    output logic Cout
);

    logic c1, c2, c3;

    // FA0
    full_adder FA0(
        .A(A[0]), .B(B[0]), .cin(Cin),
        .S(S[0]), .Cout(c1)
    );

    // FA1
    full_adder FA1(
        .A(A[1]), .B(B[1]), .cin(c1),
        .S(S[1]), .Cout(c2)
    );

    // FA2
    full_adder FA2(
        .A(A[2]), .B(B[2]), .cin(c2),
        .S(S[2]), .Cout(c3)
    );

    // FA3
    full_adder FA3(
        .A(A[3]), .B(B[3]), .cin(c3),
        .S(S[3]), .Cout(Cout)
    );

endmodule
```

Por otra parte, para el caso de la resta completa se usa una estructura muy similar al full-adder aplicada en la suma, con la diferencia que el bit proveniente del número “B” se niega y a la hora de concatenar esta nueva versión del full-adder, para el bit menos significativo se le suma un bit, esto se logra haciendo que el Cin del primer full-adder-resta, sea igual a 1, de esta forma realizamos un doble complemento:

```
module full_adder Resta(
    input logic A, // bit del número A
    input logic B, // bit del número B (se niega internamente)
    input logic Cin, // bit de entrada de acarreo
    output logic Cout, // bit de salida de acarreo
    output logic S // bit de resta
);

    logic B_neg;
    logic AxorB;
    logic and1;
    logic or1;

    // Negar la entrada B
    assign B_neg = ~B;

    // XOR entre A y B_negado
    assign AxorB = A ^ B_neg;

    // Resultado de la resta: (A * ~B) * Cin
    assign S = AxorB ^ Cin;

    // (A * ~B) AND Cin
    assign and1 = AxorB & Cin;

    // (A OR ~B)
    assign or1 = A | B_neg;

    // Carry out: (A OR ~B) OR [(A * ~B) AND Cin]
    assign Cout = or1 | and1;

endmodule

module Restador4b(
    input logic [3:0] A, // Minuendo (4 bits)
    input logic [3:0] B, // Sustraendo (4 bits)
    output logic [3:0] S, // Resultado de la resta (4 bits)
    output logic Cout // Carry de salida final
);

    logic [3:0] carry;

    // Instancia de los 4 full_adder Resta en cascada
    full_adder Resta FA0 (
        .A(A[0]),
        .B(B[0]),
        .Cin(1'b1), // Primer Cin = 1 para complemento a dos
        .S(S[0]),
        .Cout(carry[0])
    );

    full_adder Resta FA1 (
        .A(A[1]),
        .B(B[1]),
        .Cin(carry[0]),
        .S(S[1]),
        .Cout(carry[1])
    );

    full_adder Resta FA2 (
        .A(A[2]),
        .B(B[2]),
        .Cin(carry[1]),
        .S(S[2]),
        .Cout(carry[2])
    );

    full_adder Resta FA3 (
        .A(A[3]),
        .B(B[3]),
        .Cin(carry[2]),
        .S(S[3]),
        .Cout(Cout)
    );

endmodule
```

Análisis

Al realizar una simulación del ripple carry adder por medio de un test bench podemos observar que efectivamente las sumas se ejecutan correctamente con 4 bits, en caso de un acarreo se envía a la salida C_out:

1010	0011	1111	1010
0101	0101	0001	0101
1			
0000	1000	0000	
1			

De este modo confirmamos que el circuito con compuertas planteado para la suma trabaja de la forma esperada.

Referencias

[1] F. Rahman, “OS 00: Into the CPU—Registers, ALU, and Control Unit Explained,” *DEV Community*, Jul. 14, 2025. [Online]. Available: <https://dev.to/faiyaz032/os-00-into-the-cpu-registers-alu-and-control-unit-explained-3o2d>

[2] S. Harris and D. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. San Francisco, CA, USA: Morgan Kaufmann, 2021.